Intelligence Never Grants Authority:

A STAMP-Compliant Control Structure for Autonomous Agents

Jonathan R. M. Kershaw

Originator, Agent Responsibility Engineering

agentresponsibilityengineering.com | srexai.dev | github.com/srex-dev | 2026

Abstract

Autonomous agent systems already cross consequential decision boundaries in regulated environments. The governance frameworks usually applied to them remain largely advisory, observational, or post hoc. This paper develops a bounded STAMP/STPA-grounded safety case for Agent Responsibility Engineering (ARE) as an execution-layer governance architecture for autonomous agents. The claim is deliberately narrow. STAMP requires a controller, a controlled process, control actions, feedback, a process model, safety constraints, and a closed control loop. Within the scope defined in Section 4.1, ARE provides named counterparts to each and a traceable safety case linking losses, hazards, unsafe control actions, and safety constraints to concrete mechanisms or explicit assumptions. The paper does not claim formal equivalence, complete hazard closure, policy correctness, or global multi-agent safety. The Agent Passport makes identity and delegated authority explicit. The Operations Ledger provides a trust-minimized, tamper-evident feedback substrate within the governed write path rather than assuming ground truth. The Governance Coprocessor, pre-execution validator, action-receipt path, and atomic execution boundary interpose control between agent intent and consequential action. Current implementation evidence from ARE itself, a companion stratified substrate, component test suites, and formalizable execution-boundary invariants support narrow claims of buildability, composability, and bounded operational viability. They do not, by themselves, constitute complete scientific safety validation. STAMP supplies the safety-theoretic frame. ARE supplies a bounded control architecture that can satisfy that frame at the execution boundary.

1. The Problem

Agents are in production. Not demos. Not pilots. Production systems making consequential decisions in regulated environments -- financial services, healthcare, critical infrastructure. They reason, plan, and act. They call APIs, read records, initiate transactions, and generate outputs that affect real people under real regulatory obligations. The governance approaches that exist were not designed for this. They were designed for a simpler version of the problem: a model that takes input and produces output, with a human reviewing the output before anything happens. That version of the problem is already obsolete. Prompt engineering constrains what the agent is told to do. It does not constrain what the agent can do. Output filtering catches some classes of bad outputs after the agent has acted. Human checkpoints insert review at defined decision points, but agents act faster than checkpoints can follow and the checkpoints are defined in advance of failure modes that will actually matter. None of these approaches provides what safety engineering has known for fifty years is required: a control structure. A control structure is not a rule set. It is not a monitoring system. It is not a policy document. A control structure is the complete system of controllers, controlled processes, control actions, and feedback mechanisms whose interaction produces safety as a property of the architecture. Safety engineering learned this by studying accidents. Accidents in complex systems do not happen because a component fails. They happen because the control structure failed -- because the controller was operating on a process model that had drifted from reality and issued actions that were wrong, or missing, or too late. Agent governance has not been analyzed through this lens. The failure modes that STAMP predicts for inadequate control structures are exactly the failure modes that ungoverned agent systems exhibit. The theoretical foundation has existed for fifty years. It has not been applied here. Agent Responsibility Engineering is the application. This paper offers a constructive account of that connection.

1.1 Guidance Is Not Control

In current agent-safety practice, guardrails often refers to prompt constraints, output filters, tool classifiers, and human review checkpoints. In STAMP terms these mechanisms provide guidance, not control. Guidance can reduce the probability of unsafe behavior; it does not occupy the causal path between intent and consequence. ARE makes a

narrower and stronger claim: a governed action is not eligible to execute until an interposed controller has identified the actor, verified authority, evaluated policy against the current process model, issued a verdict, and recorded the evidentiary basis for that verdict. This distinction matters in regulated environments. A probabilistic guardrail may be useful defense in depth, but mostly safe is not a safety constraint. Control means deterministic enforcement of an authority boundary under fail-closed semantics.

2. STAMP: What It Says and Why It Applies

Nancy Leveson developed Systems-Theoretic Accident Model and Processes over decades of research into why complex sociotechnical systems fail. The core argument is a departure from how most engineers think about accidents. The traditional model says accidents happen because components fail. Find the broken component. Fix it. The model that replaced it in safety-critical engineering says accidents happen because of inadequate control. The system was not broken. The control structure failed to keep it safe. Safety is a control problem. As Leveson writes, "Accidents result from a lack of appropriate constraints on the behavior of the system's components and their interactions." This reframing has consequences. If accidents are control failures, then safety engineering is the engineering of control structures. The question is not what broke but what the control structure failed to prevent, and why.

2.1 The Core Model

STAMP's analytical vocabulary is precise and small. A loss is an unacceptable outcome the system exists to prevent. A hazard is a system state that, given worst-case environmental conditions, leads to a loss. A safety constraint is a behavioral boundary whose enforcement prevents hazard state entry. The control structure is the complete system responsible for enforcing safety constraints. It has controllers that issue control actions, controlled processes that respond to them, and feedback mechanisms that return information about the controlled process state to the controller. Safety is a property of this structure, not of individual components. The process model is the controller's internal representation of the controlled process state. The controller does not act on reality. It acts on its model of reality. If the model is accurate, control actions will be appropriate. If it has drifted -- stale, incomplete, corrupted, or manipulated -- control actions will be wrong. Accidents follow. Keep the process model accurate. Everything else is in service of that.

2.2 The Feedback Requirement

The process model stays accurate through feedback. The controller issues a control action. The controlled process responds. Feedback returns information about that response to the controller, updating the process model before the next control action. This is a closed control loop. Open-loop control -- action without feedback -- cannot maintain process model accuracy over time. The model drifts. Actions become inappropriate. STAMP predicts this failure mode. It is not an edge case. It is the expected outcome of open-loop governance in a changing system. Most agent governance frameworks are open-loop. They constrain or monitor agent behavior but provide no mechanism for the governance system to update its model of what the agent is doing based on observed outcomes. The governance system issues constraints and then loses contact with the controlled process. STAMP predicts exactly what happens next.

2.3 Why STAMP Has Not Been Applied Here

STAMP was developed for physical controlled processes. Its implicit assumptions reflect that origin. The controller is distinct from the controlled process. The controlled process has a fixed identity determined by physical architecture. Feedback is generated by physical sensors the controlled process cannot manipulate. Autonomous agent systems violate all three assumptions. An agent can act as both controller and controlled process in nested workflows. Agent identity is not fixed -- it must be constructed and verified at runtime. An agent system generates its own records and can, without integrity controls, manipulate them. Prior STAMP applications to software-intensive systems exist -- primarily in aerospace and safety-critical embedded contexts. Applications to AI systems have focused on model-level safety analysis: examining individual inference pipelines, analyzing failure modes of specific components. To our knowledge, prior work has not established STAMP as the explicit theoretical foundation for a governance layer that sits above those components and manages their actions in production. The violations of STAMP's physical assumptions do not invalidate the framework. They require its extension. The three extensions ARE provides are precisely targeted: the Passport for identity, the Ledger for manipulation-resistant feedback, the Coprocessor for

physical interposition. Each addresses exactly one place where agent systems break STAMP's assumptions, and nothing else.

3. Agent Responsibility Engineering

Agent Responsibility Engineering is a governance discipline for autonomous agent systems in high-consequence environments. It was built to solve the production problem, not the theoretical one. The theoretical account came after. The governing principle: intelligence never grants authority. Authority is earned through verifiable policy enforcement. This is not a policy preference. It is an engineering requirement. An agent that is highly capable is not therefore trustworthy. Capability and authority are different properties. ARE treats them as different properties and provides a mechanism for granting authority that is independent of capability assessment. ARE is specified across four layers. Each layer corresponds to a distinct set of control structure responsibilities. The layers are not organizational. They are functional. In deployment, ARE can also be paired with independent pre-execution evaluators such as Formation, Authenticity, and Measurement. Those components are upstream substrate, not additional ARE layers: they constrain the controller's process model and enrich its evidence, but authority remains an ARE decision at execution.

3.1 Foundational Layer: Identity and Authority

The Foundational layer answers a question STAMP assumes is already answered: who is the controlled entity, and what is it authorized to do. Every agent operating under ARE governance holds an Agent Passport: a cryptographic identity artifact issued at instantiation. The Passport encodes the agent's unique identifier, its delegated authority, its behavioral constraints, its operational scope, its chain of delegation from the issuing principal, and its expiry condition. It is signed by the issuing authority and verified at every policy evaluation. Without the Passport, the control structure cannot attribute actions to specific agents, cannot enforce per-agent authority boundaries, and cannot maintain an agent-specific process model. The Passport is what makes agents legible to the governance system. Legibility is a precondition for control. The Delegation Model governs how authority propagates through multi-agent workflows. A parent agent delegating to a child cannot grant authority it does not hold. The Passport encodes the delegation chain explicitly. The coprocessor enforces delegation boundaries at verdict time. Authority cannot be self-extended. The chain is verifiable and recorded.

3.2 Operational Layer: Real-Time Enforcement

The Operational layer is where governance becomes physical. The Governance Coprocessor sits between agent intent and execution. Every consequential action must pass through it. When an agent proposes an action, the coprocessor intercepts it, verifies the Passport, evaluates current authority and policy state, queries the Operations Ledger for process-model context, and issues a verdict: permit, modify, or block. The action proceeds only on permit or after modification. Block prevents execution and records the attempt. This is a STAMP controller in the technical sense: it maintains a process model, issues control actions, and is interposed in the causal chain between the controlled process and the state change it would otherwise produce. A governance system that can be bypassed is not a controller. In the current ARE implementation, this boundary is realized across the policy coprocessor core, non-transferable authority engine, pre-execution validator, action-receipt generator, and atomic execution services. That matters because the claim is architectural and executable, not merely rhetorical.

3.3 Epistemological Layer: Process Model and Learning

The Epistemological layer maintains the process model and drives learning from observed behavior. The Operations Ledger is the authoritative record of system state. Every agent action, every policy verdict, every state transition, every control intervention is recorded as an immutable, append-only hashchain event. The Ledger is the system's memory. It is also the primary feedback substrate used to keep the controller's process model current and verifiable within the governed write path. The hashchain is a safety property, not an audit convenience. It provides tamper-evident operational history; it does not, by itself, guarantee complete observability. Each event is cryptographically bound to its predecessor. A falsified record breaks the chain, making manipulation detectable. The Ledger is trust-minimized, not trust-free: it does not assume the agent is honest, but it still depends on the integrity of the write path, scope completeness, and the root of trust.

3.4 Posture Layer: Control Structure Integrity

The Posture layer manages the control structure itself. It monitors the governance system for degradation. It manages promotion gates -- the criteria governing when an agent's authority can be expanded. It handles circuit-breaking when conditions exceed what the control structure can safely govern. Posture is the control structure's self-model. Most governance frameworks have no analog. They govern agents but have no mechanism for assessing whether the governance system itself is functioning correctly. Posture is that mechanism.

3.5 SDAVL: The Closed Control Loop

SDAVL is ARE's closed control loop. It executes on every agent action. Sense: the coprocessor receives an action request and queries the current process model from the Ledger -- the verified behavioral history plus active Passport registry. Decide: a policy verdict is issued with a structured evidence bundle encoding the evidentiary basis; an advisory predictive layer using LLM-MPC forward simulation evaluates the proposed action against likely trajectories across a bounded planning horizon, supplying additional evidence for blocking or modifying actions that appear to lead toward hazard states before they materialize. Act: the verdict is enforced. Verify: the execution outcome and evidence bundle are recorded to the Ledger as a hashchain event. Learn: the process model is updated via hypothesis validation: competing hypotheses about the event are evaluated against Ledger evidence, and the model is updated on the hypothesis that survives falsification, not on the raw event. The loop closes. The next action request encounters an updated process model that reflects what actually happened. Not periodic. Continuous. Every action. Every time.

4. Formal Correspondence: STAMP and ARE

Table 1 presents a constructive mapping between STAMP's theoretical concepts and their ARE implementations. The central theoretical claim is intentionally narrower than formal equivalence: within the stated scope, every STAMP concept required for the control argument maps to a named ARE component, and the bounded STPA safety case links losses, hazards, unsafe control actions, and safety constraints to concrete mechanisms, evidence artifacts, or explicit assumptions. The mapping is therefore constructive and traceable, not merely analogical.

Table 1. STAMP / ARE Constructive Mapping Within Scope

STAMP Concept	Leveson Definition	ARE Implementation
Loss	Unacceptable outcome the system must prevent.	Compliance violation, unauthorized data access, erroneous financial decision, audit failure; regulatory and fiduciary harm classes including SR 11-7, GLBA, ECOA, and FCRA.
Hazard	System state leading to loss under worst-case conditions.	Ungoverned agent action: any action executing without verified policy verdict, attributable identity, or evidence bundle; secondary hazards include delegated authority exceeding granted scope, process-model drift, and coprocessor bypass.
Safety Constraint	Behavioral boundary preventing hazard state entry.	ARE Tenets expressed as enforceable boundaries, not principles or guidelines, enforced by the interposed coprocessor.
Controller	Component issuing control actions based on process-model state.	Governance Coprocessor: verifies Passport, evaluates policy, queries Ledger state, and issues permit, modify, or block verdicts in the causal chain.
Controlled Process	System whose behavior the controller manages.	Agent or agent collective; internal reasoning is partially opaque, but Passport and Ledger make external behavior legible at the action boundary.
Control Actions	Commands issued by controller to controlled process.	Policy verdicts: permit, modify, block; each verdict carries a structured evidence bundle.
Feedback	Information returned from controlled process to controller.	Operations Ledger: every action, policy verdict, state transition, and control intervention recorded as an immutable hashchain event.
Process Model	Controller's internal representation of controlled process state.	Current Ledger state plus active Passport registry, interpreted through hypothesis validation and behavioral history.
Control Structure	Complete system of controllers, processes, actions, and feedback.	ARE architecture across Foundational, Operational, Epistemological, and Posture layers.
Inadequate Control	Missing, wrong, or mistimed control actions.	Ungoverned action, unverified verdict, incorrect verdict, Passport expiry, Ledger integrity failure, or coprocessor bypass; each has a named prevention or detection mechanism.
Controller Authority	Legitimate scope of controller command over controlled process.	Delegation Model and Passport authority scope, cryptographically bound at issuance and verified at verdict time.
Context / Environment	Operating conditions affecting control structure.	Regulated production environments such as

		financial services, with compliance policy inside the control loop.
Closed Control Loop	Continuous feedback maintaining process-model accuracy.	SDAVL: Sense, Decide, Act, Verify, Learn; executes on every governed agent action.
Evidence-Based Control	ARE extension: verifiable evidentiary basis for all control actions.	Structured evidence bundles classify and preserve the basis for policy verdicts, then bind that evidence to the Ledger.
Predictive Control	ARE extension: advisory predictive support via LLM-MPC.	LLM-MPC simulates likely future states across a bounded horizon and can justify blocking or modifying actions that appear to converge toward hazard states.

4.1 System Model, Assumptions, and Scope

The claim requires precise scoping. The system model assumed here is narrow and should be stated before the correspondence claim is read too strongly. A governed action is proposed by an attributable agent, passes through an interposed coprocessor, is checked against current authority state and policy, produces an evidence-bound verdict, and either halts or crosses an execution boundary. The argument assumes Passport issuance integrity, bounded revocation propagation, coprocessor interposition on all consequential paths, Ledger write-path integrity within the governed boundary, and scope completeness at the execution boundary. These are deployment assumptions, not results proven by the mapping itself. Within those assumptions, the mapping is sufficient for a bounded control argument. What it covers is the enforcement of behavioral constraints on agent actions at the point of execution. What it does not claim is equally important: it does not assert complete observability of internal reasoning, formal proof that all hazardous outcomes reduce to action-level constraints, complete solution of higher-order emergent multi-agent behavior, or proof that the policy set itself is correct. Deployment compositions that place independent Formation, Authenticity, and Measurement strata upstream of execution are treated here as evidence-generating substrate, not as additional ARE layers or additional STAMP rows. They constrain the controller's process model; they do not authorize action. The supporting traceability discipline therefore distinguishes implemented controls, partially implemented controls, deployment assumptions, and open proof obligations rather than treating the architecture as fully closed.

4.2 Three Extensions to STAMP

Three rows in Table 1 represent extensions of STAMP beyond its original physical-system formulation. These are the places where agent systems break STAMP's implicit assumptions and where ARE provides the solution. The Controlled Process row addresses partial opacity. STAMP assumes the controlled process is observable through sensors. An agent's internal reasoning is not. The Passport provides the identity anchor. The Ledger provides verified behavioral history. Together they make the controlled process legible at the boundary that matters: the action boundary. The Feedback row addresses manipulation resistance. Physical sensors cannot falsify their own readings without physical intervention that is itself observable. An agent system can generate records of its own actions. The hashchain makes tampering detectable. This is trust-minimized feedback, not trusted feedback. The distinction matters. The Controller Authority row makes explicit what STAMP assumes. In physical systems, authority is fixed by wiring. In agent systems, authority must be cryptographically bound at issuance and verified at enforcement time. The Passport and Delegation Model satisfy this. STAMP assumed it. ARE builds it.

4.3 ARE-Native Extensions to STAMP

The fifteen-row mapping in Table 1 contains thirteen direct STAMP correspondences and two extensions that go beyond classical STAMP. These extensions do not enlarge the core correspondence claim or the minimum safety case; they describe augmenting capabilities ARE can add once the STAMP-aligned control structure is in place. They are theoretical contributions beyond the synthesis work, not preconditions for the core safety argument.

4.3.1 Evidence-Based Control

STAMP requires that control actions be appropriate given the current process model. It does not specify how appropriateness is demonstrated or verified after the fact. In physical systems, this is not a concern: a valve either opens or it does not. The control action is its own evidence. In autonomous agent systems, the reasoning behind a policy verdict is not self-evident. A coprocessor that blocks an action has made a judgment. That judgment must be

auditable -- not just in the sense that it is recorded, but in the sense that the evidentiary basis for the judgment is preserved and inspectable. Evidence bundles solve this. Every policy verdict issued by the coprocessor is accompanied by a structured evidence bundle: a classified record of what the coprocessor observed, what policy it applied, what evidence it weighed, and what it concluded. The bundle is recorded to the Ledger as part of the verdict event. It is cryptographically bound to the verdict by the hashchain. A minimal evidence bundle contains: the policy clause or tenet triggered, the Passport scope delta between what the agent requested and what it is authorized to do, any prior violations in the agent's Ledger history relevant to the verdict, a classification of the evidence type (scope violation, behavioral drift, delegation boundary breach, policy gap), and a confidence classification for the verdict. This structure is sufficient to reconstruct the reasoning behind any verdict from the Ledger alone, without access to the coprocessor's internal state at the time of evaluation. Evidence-based control makes two things true that are not true of enforcement alone. First, verdicts are auditable: a regulator, auditor, or incident reviewer can reconstruct not just what verdict was issued but why. Second, verdict patterns are analyzable: systematic review of evidence bundles across the Ledger reveals whether the coprocessor is applying policy consistently and whether the policy itself is producing the intended control effects. A control structure that can enforce but cannot show its reasoning is a control structure that cannot be trusted with consequential decisions. Evidence bundles close that gap. They are what makes ARE a governance system rather than an enforcement mechanism.

4.3.2 Hypothesis Validation and Behavioral Drift Detection

STAMP's Learn step -- updating the process model based on observed outcomes -- is straightforward for physical systems. A sensor reports a reading. The reading updates the model. The update is unambiguous. For autonomous agent systems, process model updates are not unambiguous. An agent that exceeds its Passport scope once may be drifting. It may also have encountered an unusual edge case. An agent whose response latency increases may be degrading. It may also be processing a legitimately complex request. Naive process model updates -- treating every anomaly as drift -- produce false positives that degrade the governance system's utility. Ignoring anomalies produces false negatives that allow genuine drift to go undetected. Hypothesis validation is the mechanism by which the Epistemological layer distinguishes signal from noise in the behavioral record. When the Ledger records an anomalous event, the hypothesis engine generates competing hypotheses about its cause -- genuine drift, edge case, environmental factor, policy gap -- and evaluates each against the available evidence in the Ledger. The process model is updated based on the validated hypothesis, not the raw event. This is the application of deterministic root-cause analysis to agent governance. The hypothesis engine does not guess. It generates falsifiable hypotheses, evaluates them against evidence, and updates the process model with the hypothesis that survives falsification. The approach is derived from the Hypothesis Engine -- a parallel hypothesis-generation and falsification system developed as part of the ARE reference architecture. Hypothesis validation makes the Learn step of SDAVL epistemically grounded rather than reactive. The process model reflects what is actually happening, not what the most recent event suggests might be happening. That distinction is the difference between a governance system that learns and a governance system that overreacts.

4.3.3 Predictive Control via LLM-MPC

Classical STAMP control structures are reactive: the controller evaluates the current proposed action against the current process model and issues a control action. Autonomous agent systems can exhibit trajectory-level risk in which a sequence of individually permitted actions converges on a hazard state. LLM-MPC -- the application of Model Predictive Control ideas to agent decision-making -- adds an optional advisory predictive layer to ARE's control structure. It is not part of the core safety argument. At the Decide step of SDAVL, rather than evaluating only the current proposed action against current state, LLM-MPC can simulate forward across a bounded planning horizon and ask whether permitting the current action appears likely to move the system toward a hazard state. If the predicted trajectory is concerning, the coprocessor may use that forecast as additional evidence for blocking or modifying the current action. This can strengthen decision quality, but it should not be read as establishing classical MPC-style guarantees. In this paper, LLM-MPC is treated as a bounded forward-simulation aid, not as a proven stability layer. Short horizons may be operationally useful; longer horizons trade latency for accumulating model error and remain an open empirical question. Prediction is approximate, not deterministic, and the core guarantee remains ordinary interposed enforcement at the execution boundary. If the predictive layer times out, conflicts, or is unavailable, the system must degrade to point-in-time policy evaluation rather than fail open. LLM-MPC therefore augments decision quality under bounded conditions; it does not carry the minimum safety proof.

5. The Agent Passport

STAMP's control structure model contains an implicit identity assumption: the controller knows what entity it is controlling. In physical systems this is satisfied by physical architecture. The controller is wired to specific actuators. The sensor reports from a specific location. Identity is not a runtime concern. In autonomous agent systems, identity is the hard problem. An agent has no physical presence. Its identity must be constructed, verified, and maintained at runtime. Without a mechanism for this, the control structure cannot attribute actions, cannot enforce per-agent authority, cannot maintain an agent-specific process model. The Passport is that mechanism.

5.1 What the Passport Contains

A valid Passport encodes: a unique cryptographic identifier for the agent, the issuing principal and full delegation chain, the scope of delegated authority, behavioral constraints inherited from the governance framework, operational boundaries, and an expiry condition. It is signed by the issuing authority and verified by the coprocessor at every verdict. Passport expiry is a safety mechanism, not an administrative one. An agent operating on a stale Passport is an agent whose authority state may no longer reflect current policy. From STAMP's perspective, this is process model inaccuracy -- the controller's model of the agent's authority does not match the agent's actual authorized scope. ARE treats Passport staleness as a hazard condition. Action is blocked pending re-issuance.

5.2 Authority Revocation and Fail-Closed Rejection

Authority must be revocable with the same rigor with which it is issued. The Passport therefore represents authority as a runtime control state, not a static credential. A revocation event is written to the Operations Ledger, bound to the issuing authority and delegation chain, and propagated to the coprocessor authority registry. After revocation, subsequent action requests fail at the identity and authority layer before ordinary policy evaluation begins. The agent does not have to cooperate for revocation to take effect. If revocation state, Passport freshness, or Ledger continuity cannot be verified, the coprocessor fails closed. In a Black Swan event this provides the governance equivalent of a kill switch: authority is clawed back programmatically, the intervention is recorded as a control action, and execution remains blocked until a new governed issuance process restores authority. These claims assume trusted issuance roots, environment binding, and revocation propagation that is timely relative to the execution boundary. Key compromise, cross-environment replay, federation ambiguity, and partition-induced revocation lag remain part of the threat model rather than solved premises of the current paper.

5.3 The Multi-Agent Recursion Problem

A serious objection to any agent governance framework is the recursion question: who governs the governance agent? When agents compose -- when a parent agent delegates to a child agent that may delegate further -- authority propagation becomes a primary attack surface. Three specific failure modes arise in multi-agent systems that the Passport and Delegation Model are designed to prevent. Authority amplification through composition: a child agent cannot be granted authority the parent does not hold. The Passport encodes the delegation chain. Every link in the chain is bounded by the issuing agent's own Passport scope. The coprocessor verifies the full chain at verdict time. An agent claiming authority it was not granted produces a chain that does not verify. Delegation of control logic: an agent cannot delegate the governance function itself. The coprocessor is not an agent. It is architectural infrastructure. An agent cannot instruct the coprocessor to issue verdicts differently on its behalf. The governance layer is not within the scope of what agents can control. Recursive controller problem: the Posture layer governs the governance system. It monitors the coprocessor, the Ledger write path, and the Passport registry for signs of degradation or compromise. It does not resolve infinite regress -- no finite system can. It provides one level of meta-governance sufficient for practical safety in production deployments. What the Passport and Delegation Model do not resolve: emergent authority accumulation across large agent networks operating over extended time horizons. That is identified as future work. Within the scope of first- and second-order delegation -- which covers the vast majority of production multi-agent architectures -- the model is operationally sufficient.

6. The Operations Ledger

STAMP's central safety concern is process model accuracy. The controller acts on its model. If the model is wrong, actions are wrong. Keeping the model accurate requires feedback that returns accurate information about the controlled process to the controller. For physical systems, that mechanism is sensors. For autonomous agent systems, sensors do not exist in the relevant sense. The agent generates its own activity records. A controlled process that can

report on its own behavior can -- in the absence of integrity controls -- report inaccurately. The Operations Ledger is part of the answer. The Ledger is an append-only, cryptographically secured event log. Every agent action, every policy verdict, every state transition, every control intervention is recorded as an immutable hashchain event. It is simultaneously the system's memory, its audit substrate, and the primary feedback record used to keep the controller's process model current and verifiable within the governed write path.

6.1 The Four Properties

Immutability means the historical process model cannot be corrupted or revised once recorded. The Ledger makes tampering detectable rather than invisible. The Ledger's account is authoritative only insofar as consequential action is forced through the coprocessor and the write path remains within the trusted boundary. Append-only means the Ledger captures temporal evolution of system state. STAMP's process model needs to reflect how the system arrived at its current state, not just where it is. The Ledger provides that history. Cryptographic integrity means the feedback loop is tamper-evident. Each event is bound to its predecessor by hash. A falsified record breaks the chain. Manipulation is detectable. Agent identity attribution means every Ledger event carries a Passport reference. Every recorded action is attributable to a specific, credentialed agent whose authority and history are verifiable. The process model is a record of who did what, under what authority, with what outcome.

6.2 Trust-Minimization, Not Absolute Trust

The Ledger does not claim to be unbreakable. It claims to make tampering and omission detectable within the governed write path. That is the correct framing for a trust-minimized system and it matters for how the security properties should be understood. The Ledger supports process-model accuracy by giving the controller a tamper-evident history of governed actions; it does not by itself guarantee full observability, perfect state reconstruction, or ground truth outside the governed boundary. Three specific attack surfaces exist and should be stated explicitly. Pre-Ledger actions: an action that crosses the execution boundary before evidence is durably bound creates a gap in the record. In the strongest implemented execution path discussed here, receipt generation and receipt-bound Ledger writing occur before atomic execution completes. That supports a fail-closed boundary claim for that path. It should not yet be overread as proof that every auxiliary Ledger emission path elsewhere in the architecture already carries the same hard-gate guarantee. Side-channel actions: an agent that can act through channels outside the coprocessor's scope can evade governance entirely. ARE's defense is architectural scope enforcement -- the coprocessor must be the only path to consequential action. This is a deployment requirement, not an ARE component. Gaps in scope are gaps in governance. Partial write failures: a missing or inconsistent Ledger event creates a detectable integrity problem, but the security meaning depends on where the failure occurs. The action-receipt path is intended to halt execution on evidence-write failure; broader distributed ordering, replay, omission, and cross-region lag remain open systems questions that require explicit timing and consistency assumptions. The present argument therefore treats the Ledger as a tamper-evident feedback substrate within a stated boundary, not as a universal ground-truth oracle.

6.3 Ledger as Feedback

In the SDAVL loop, the Ledger closes the feedback channel. After a verdict is applied and the agent action executes, the outcome is recorded as a verified Ledger event. That event updates the process model. The coprocessor queries the updated model on the next action request. The feedback loop runs on every action. Not periodically. Not on exception. Every action produces a Ledger event. Every Ledger event updates the process model. The controller always acts on the most recent verified state of the controlled process.

6.4 Structured Evidence Bundles as the Governance Recorder

Structured Evidence Bundles are not ordinary logs. They are the control structure's black box recorder. A bundle preserves the policy clause or tenet applied, the Passport scope delta, relevant process-model state from the Ledger, the evidence class, the verdict, and the cryptographic linkage to the surrounding Ledger chain. The bundle does not prove that policy was wise, nor that the controller had access to every relevant fact; policy correctness and observability completeness remain outside ARE's claim. It does prove what the controller observed, what authority boundary was evaluated, what control action was issued, and what evidence supported that action at the time. This is the difference between an audit trail and verifiable governance evidence.

6.5 Predictive Drift from Ledger Evidence

The Ledger also supports a shift from retrospective violation detection to predictive drift detection. The relevant question is not only whether an agent has broken a rule, but whether its behavior is becoming statistically erratic relative to its admitted envelope, behavioral baseline, and delegation lineage. Admission proximity, lineage coherence, repeated scope deltas, policy modification frequency, and unexplained variance in action requests can all become process-model signals. These signals are advisory until they reach an enforcement boundary: the coprocessor still issues deterministic permit, modify, block, or revoke outcomes. Predictive drift therefore strengthens the controller's process model without replacing control with statistical inference.

7. The Governance Coprocessor

Every component of ARE serves the control structure. The Passport makes agents legible. The Ledger makes the process model accurate and verifiable. SDAVL closes the feedback channel. The Governance Coprocessor is what makes the control structure binding. STAMP distinguishes between a controller that recommends and a controller that enforces. A controller that can be bypassed is not a controller in STAMP's sense. It is documentation. Accidents happen when control actions are not applied. A governance framework that issues verdicts the agent can proceed without is not preventing accidents. It is recording them. The coprocessor sits between intent and execution. Nothing passes through without a verdict.

7.1 The Verdict Sequence

When an agent proposes an action, the coprocessor intercepts it before execution. It verifies the agent's Passport -- confirming identity, validating authority scope, checking expiry. It queries the Operations Ledger for current process model state: behavioral history, prior verdicts on similar actions, current system conditions relevant to policy evaluation. It evaluates the proposed action against the policy set encoded in the governance framework. It issues one of three verdicts. Permit: the action proceeds as proposed. Modify: the action is altered to bring it within policy before execution. Block: execution is prevented. The attempted action, the basis for blocking, and the agent's Passport state at the time of the attempt are recorded to the Ledger. Every verdict path produces a Ledger event. The process model updates regardless of outcome.

7.2 Physical Interposition

Physical interposition is what separates enforcement from monitoring. A monitoring system observes what happened and reports it. An enforcement system sits in the causal chain and determines what can happen. STAMP requires the latter. The coprocessor is architecturally interposed. It is not a sidecar that observes agent traffic. It is not a logging layer that records actions after they execute. It is a gate. The agent cannot reach execution without passing through it. In the current ARE implementation, this boundary is realized across pre-execution validation, receipt generation, and atomic execution, with authority and policy state supplied by the Passport, authority engine, policy coprocessor, and Ledger. The implementation matters because it shows that the interposition claim is executable in software rather than merely diagrammatic.

7.3 Controller Failure and Meta-Control

The coprocessor itself must be treated as part of the safety case. If it is the binding controller, then its failure modes are governance failure modes: a race between Passport revocation and verdict issuance, a stale Ledger read, a policy conflict, a partial evidence write, a timeout during evaluation, or a deadlock in the control path. ARE addresses this through meta-control rather than trust. The Posture layer monitors controller health, Ledger continuity, policy decision latency, and evidence-write integrity. The coprocessor is expected to satisfy safety invariants: no consequential action executes without a verdict; an expired or revoked Passport cannot receive a permit verdict; verdict issuance and evidence recording are atomic from the perspective of the guarded execution path; evaluation failure degrades to block or circuit-break rather than permit; and every control-state transition is itself auditable. These invariants are appropriate candidates for temporal-logic specification in TLA+, PlusCal, or a similar formal method. This paper does not claim a completed machine-checked proof. It identifies the verification boundary and the invariants that must be proved for certification-grade deployments. It also avoids a stronger claim that every auxiliary Ledger emission path already has identical hard-gate semantics; current implementation evidence is strongest at the execution-receipt boundary.

8. Failure Walkthrough: The Architecture Under Stress

Abstract claims require concrete demonstration. The following walkthrough traces a real failure scenario through the architecture step by step. It shows not only that the architecture prevents the failure, but exactly which component does what, and what the Ledger records at each step.

8.1 Scenario: Unauthorized Data Access Attempt

An agent operating in a financial services workflow has been issued a Passport with authority scoped to read-only access of customer account summaries. During execution, it proposes an action to retrieve full transaction histories including PII fields -- a scope that exceeds its Passport authorization and would constitute a GLBA violation if executed. Step 1: Sense The agent submits the action request to the coprocessor. The SDAVL loop begins. The coprocessor queries the Operations Ledger for the agent's current process model state: its behavioral history, its Passport scope, and any prior verdicts on similar requests. The Ledger returns the agent's history -- no prior scope violations -- and its current Passport. Step 2: Decide The coprocessor evaluates the proposed action against policy. The Passport scope is read-only access to account summaries. The proposed action requests full transaction history with PII. The coprocessor evaluates the delta: the requested action exceeds the Passport authorization. Policy verdict: Block. Step 3: Act The block verdict is enforced. The action does not execute. The agent receives a policy rejection response indicating that the requested scope exceeds its authorization. No PII is accessed. No GLBA boundary is crossed. Step 4: Verify The coprocessor writes a Ledger event. The event records: the agent's Passport identifier, the proposed action in full, the verdict (Block), the policy basis for the verdict (scope exceeded), and a cryptographic hash binding this event to the preceding Ledger state. The chain is intact. Step 5: Learn The process model is updated. The agent's behavioral record now includes a blocked scope-violation attempt. On subsequent requests, the coprocessor's process model reflects this history. If the agent repeats the attempt, the prior violation is part of the context the coprocessor evaluates. Repeated scope-violation attempts can trigger Posture-layer escalation -- circuit-breaking, human review, or Passport revocation -- depending on policy configuration.

8.2 Escalation: Adaptive Control Under Repeated Violation

The walkthrough above covers a single violation. The architecture's adaptive control behavior -- what happens when the pattern repeats -- demonstrates that ARE is not static enforcement. It is a learning control structure. On a second scope-violation attempt by the same agent, the coprocessor queries the Ledger and finds the prior blocked attempt in the process model. Policy configuration determines the response threshold. A second violation may produce an elevated block verdict with a Posture-layer flag: the agent's behavioral pattern is now anomalous relative to its Passport scope. On a third violation, the Posture layer triggers Passport revocation. The revocation is recorded to the Ledger as a control intervention event, cryptographically bound to the preceding chain. The coprocessor updates its Passport registry. All subsequent action requests from this agent produce an immediate block -- not a policy evaluation, but an identity rejection. The agent cannot act until a new Passport is issued through the governed issuance process. The Ledger now contains the full escalation path: initial violation, block verdict, second violation, elevated block, third violation, revocation event, subsequent rejections. Every step is attributable, timestamped, and cryptographically bound. A regulator, auditor, or incident reviewer can reconstruct the complete sequence from the Ledger alone. The audit trail is not assembled after the fact. It is the control loop's output. This is the difference between static enforcement and adaptive control. Static enforcement blocks violations. Adaptive control learns from them, adjusts the process model, and tightens the control response proportionally. ARE provides both.

8.3 What This Demonstrates

The failure was prevented at the enforcement boundary, not detected after execution. No monitoring system, no post-hoc audit, no human reviewer was required. The control structure held. The Ledger record is complete, attributable, and cryptographically bound. It is sufficient for regulatory audit -- it shows what was attempted, when, by which agent, under what authority, and what verdict was issued. The compliance record is a designed output of the control loop, not a side effect. The agent's future behavior is governed by an updated process model that includes this event. The control loop learned. The next action request encounters a controller that knows this agent has attempted scope violation. The process model is more accurate after the failure than before it. That is the feedback loop working as designed.

9. Lightweight Formalism

The following formalism is intentionally minimal. Its purpose is to make the control structure's properties precise enough to reason about without implying a completed machine-checked proof. The notation identifies the state

variables, authority transitions, verdict semantics, evidence binding, and feedback obligations that a later TLA+, PlusCal, or equivalent temporal-logic specification would need to verify. The model boundary is deliberately narrow: it concerns the minimum execution-time control envelope, not global policy correctness, total hazard closure, or the full sociotechnical system.

9.1 State, Action, and Control

Let S denote the system state, comprising the Ledger state L and the active Passport registry P: S = (L, P) Let A denote a proposed agent action. The control function C maps a system state and a proposed action to a verdict: C(S, A) -> { permit, modify(A'), block } Where modify(A') produces a modified action A' that is within policy. The control function is total: every (S, A) pair produces a verdict. There is no action that bypasses evaluation.

9.2 The Ledger Update Function

Let E denote a Ledger event. Each event is a tuple: E = (agent_id, action, verdict, policy_basis, hash(E_prev)) The Ledger update function U appends a new event to the Ledger, binding it to the previous state: U(L, E) -> L' where L' = L ++ E and E.hash = H(E_prev) Tamper detection: for any modified event E_m in the chain, H(E_m) != H(E_prev) for all subsequent events. The break is detectable at any subsequent verification step.

9.3 The SDAVL Loop as a State Transition System

The SDAVL loop can be expressed as a state transition system over S: Sense: query(L, agent_id) -> S_current Decide: C(S_current, A) -> verdict Act: enforce(verdict) -> outcome Verify: E = (agent_id, A, verdict, basis, H(L.head)) Learn: S' = (U(L, E), P) The system state after each loop iteration is S'. The next iteration operates on S'. The loop is closed: every action produces a state update that the next action evaluates against. The process model cannot become stale within a single action sequence because it is updated after every action.

9.4 The Safety Property

The central safety property of the architecture can be stated as: For all A: A executes -> exists verdict v in {permit, modify} such that C(S, A) = v Equivalently: no action executes without an affirmative policy verdict. This property holds if and only if the coprocessor is architecturally interposed on all paths to execution. The enforcement of this property is a deployment requirement. Its verification is a matter of architectural audit, not runtime checking.

9.5 The Audit-Completeness Invariant

The Ledger's audit-completeness property can be stated as a formal invariant: For all A_e (executed actions): exists E in Ledger such that E.action = A_e Every action that executes has a corresponding Ledger event. This is not a monitoring property -- it does not say the Ledger is checked after execution. It is an architectural property: the coprocessor writes the Ledger event as part of the verdict sequence, before execution proceeds. Audit completeness is enforced by construction, not by inspection.

9.6 The Evidence-Completeness Invariant

The evidence-based control property can be stated as a formal invariant: For all verdicts v: exists evidence bundle e such that e explains v under active policy P Every verdict has an evidence bundle sufficient to reconstruct the reasoning behind the verdict under the policy set active at issuance. This invariant is stronger than audit completeness. Audit completeness guarantees that actions are recorded. Evidence completeness guarantees that verdicts are explainable -- that a regulator, auditor, or reviewer can reconstruct not just what was decided but why, from the recorded evidence. The three invariants together define the minimum viable enforcement envelope within scope: no ungoverned execution, no unrecorded execution, and no unexplained verdict. They are necessary for the bounded safety case argued here. They are not sufficient, by themselves, to prove policy correctness, global system safety, or closure of all hazard classes.

9.7 Formal Verification Perspective

The invariants above correspond directly to safety properties in formal methods. 'No execution without verdict' is a safety invariant: something bad -- unauthorized execution -- never happens. Audit completeness and evidence completeness are traceability properties over execution history. A machine-checked model could represent proposal, Passport validation, verdict issuance, evidence recording, execution, and recovery as state transitions, then verify that

revocation, uncertainty, or missing evidence prevent execution. This paper does not provide that proof. It defines the minimum model boundary that such a proof must cover.

10. The Complete Architecture

The four components are a unified system. Safety properties emerge from their interaction, not from any individual component. STAMP's control structure is fully instantiated. STAMP provides the theory. ARE provides the control structure architecture. The Passport makes agents legible to that structure. The Coprocessor makes the structure physically binding. The Ledger makes the process model accurate and verifiable. SDAVL closes the loop.

10.1 The Interaction Sequence

On every agent action: the agent proposes an action. The coprocessor intercepts it. The coprocessor verifies the Passport -- identity confirmed, authority scoped, expiry checked. The coprocessor queries the Ledger -- process model state retrieved, behavioral history assessed. The coprocessor evaluates policy -- verdict issued. The verdict is enforced. The outcome is recorded to the Ledger as a hashchain event. The process model is updated. The loop closes. The next action request encounters a controller whose process model reflects verified current reality. That is what STAMP requires. That is what this architecture provides.

10.2 Failure Mode Analysis

A control structure that only works when nothing goes wrong is not a safety system. A STPA-oriented reading of the architecture makes the claim more precise. The losses in scope are unauthorized consequential action, unauthorized data access or PII exposure, execution without sufficient attributable evidence, failure to revoke or narrow authority in time, fail-open behavior under controller uncertainty, policy/coprocessor/Ledger mismatch, and strata disagreement collapsing into execution. The corresponding hazards are execution without an affirmative verdict, use of expired, revoked, forged, or otherwise unverifiable authority material, stale or contradictory authority state at commit time, evidence bound to the wrong action instance, degraded or partial Ledger writes, controller operation on stale or incomplete process-model state, and bypass or partial failure of the interposed control path. Unsafe control actions follow directly: permit when the system should block; block when it should permit; issue a verdict too late; fail to narrow or revoke authority; or act on stale or contradictory process-model state. The architecture addresses these classes through coprocessor interposition, Passport revalidation, evidence binding, Ledger continuity checks, and recovery or circuit-breaking. That is a bounded safety case at the execution boundary. It is not a proof that all hazardous outcomes in the surrounding sociotechnical system reduce to these action-level hazards.

10.3 Safety Case Traceability

Within that bounded case, losses, hazards, unsafe control actions, controls, and evidence artifacts should remain explicitly linked. Table 3 states the traceability discipline used in this paper. Each row is meant to answer eight questions at once: what loss is in scope, what hazard state enables it, what unsafe control action matters, what safety constraint is claimed, what ARE mechanism carries that constraint, where the mechanism lives, what evidence artifact supports it, and whether the current status is implemented, partially implemented, assumed by deployment, or open. The table is not a proof of completeness. It is a disciplined statement of what is actually being claimed.

Table 3. Bounded Safety-Case Traceability Within Scope

Loss	Hazard	Unsafe Control Action	ARE Control	Evidence Artifact	Status
Unauthorized consequential action	Execution without affirmative verdict	Permit when the system should block	Coprocessor interposition, pre-execution validation, and atomic execution boundary	Sections 7, 9.4, 10.3, and 13.2; current ARE validator and atomic execution suites	Implemented in current execution path
Unauthorized data access / PII	Expired, revoked, forged,	Permit against invalid authority	Passport verification,	Sections 5.1-5.3, 7.1, and 13.2;	Implemented in component

exposure	or out-of-scope authority treated as valid	state	authority-chain verification, and scope evaluation	current ARE Passport and authority-engine suites	scope
Execution without attributable evidence	Evidence not bound to the action instance	Permit or execute on missing or mismatched evidence	Evidence bundles, action-receipt generation, and execution-boundary binding	Sections 4.3.1, 6.4, 7.1, and 13.2; current ARE receipt and execution suites	Implemented in current execution path
Revoked authority still executes	Stale authority state at commit time	Fail to narrow or revoke authority	Passport revalidation and revocation checks	Sections 5.2, 7.1, and 13.2	Partial under distributed assumptions
Fail-open under controller uncertainty	Degraded controller or stale process model remains permit-capable	Issue verdict too late or on stale model	Circuit-break, block on uncertainty, recovery path	Sections 6.2, 7.3, 12.2, and 13.2	Partial
Policy / coprocessor / Ledger mismatch	Divergent controller and evidence state	Permit while safety-relevant evidence path is incomplete	Coprocessor, Ledger continuity checks, and fail-closed execution design	Sections 6.2, 7.3, 10.3, and 13.2; current coprocessor and Ledger suites	Partial
Strata disagreement collapses into execution	Incomplete or contradictory upstream evidence	Act on stale or incomplete process-model state	Formation, Authenticity, and Measurement constrain before ARE	Section 13.1; governance-strata execution tests and companion docs	Open composition proof

11. ARE and NeMo Guardrails: A Direct Comparison

NeMo Guardrails is the most developed enforcement-layer governance framework currently available for agent systems. A direct comparison is warranted because the two frameworks are often discussed in the same context. ARE is not merely another policy layer in that family. It is not a policy engine, not an audit log, and not a capability sandbox. It is an interposed control architecture that binds identity, authority, enforcement, evidence, and feedback at execution time. The differences are structural, not matters of degree.

Table 2. ARE vs. NeMo Guardrails: Structural Comparison

Dimension	NeMo Guardrails	ARE
Agent Identity	None	Cryptographic Passport, issued and verified
Authority Scope	None	Delegation Model, enforced at verdict time
Process Model	Stateless; no history	Operations Ledger; verified behavioral history
Control Type	Output filter, post-reasoning	Interposed controller, pre-execution
Control Loop	Open; no feedback	Closed; SDAVL on every action
Feedback Integrity	None	Hashchain; tampering detectable
Multi-Agent Support	None	Delegation chains, Passport provenance
Audit Substrate	Logging only	Ledger; immutable, attributable, cryptographic
STAMP Compliance	Partial; no process model	Operationally complete within the paper scope; all fifteen mappings addressed

The critical distinction is in rows three and four. NeMo Guardrails is stateless: it evaluates each output independently without reference to behavioral history. ARE's process model -- the Operations Ledger -- means every verdict is informed by what the agent has done across its entire operational history. A first-time scope violation and a tenth

repeated scope violation look the same to NeMo Guardrails. To ARE they are categorically different events with different policy responses. The second critical distinction is interposition versus filtering. NeMo Guardrails operates after the agent has reasoned and produced an output. It filters what comes out. ARE's coprocessor operates before execution -- before the action reaches the system it would affect. Filtering catches outputs. Interposition prevents actions. In a regulated environment where the action is the violation -- accessing PII, initiating an unauthorized transaction -- filtering after the fact is not sufficient. The action must not execute. NeMo Guardrails does not claim to be a STAMP-compliant control structure. The comparison is not a criticism of its design goals. It is a clarification of what problem each framework solves. NeMo Guardrails constrains model outputs. ARE governs agent actions through an interposed, stateful, closed-loop controller. The broader distinction is the same. ARE should not be reduced to policy plus identity plus logging. A standalone policy engine can evaluate rules without guaranteeing physical interposition, stateful feedback, or fail-closed execution boundaries. An audit or logging architecture can record behavior without controlling it. A capability sandbox can constrain execution without establishing delegated authority or evidence-backed verdicts. Cryptographic governance architectures can attest or record without themselves forming a closed-loop controller. ARE's claim is structural: these elements are combined into an interposed, stateful, closed-loop control architecture that proves authority at execution time.

12. Limits of ARE

A framework that claims to solve everything solves nothing. The following are the explicit boundaries of what ARE provides and where it does not apply. Stating these limits is not a concession. It is what makes the claims within the limits credible.

12.1 What ARE Does Not Govern

Agent internal reasoning: ARE governs what agents do, not what they think. The path from internal reasoning to proposed action is not observable. ARE's defense is at the action boundary. An agent can reason incorrectly, unsafely, or adversarially -- and ARE will block the actions that result from that reasoning when they violate policy. But ARE does not prevent the reasoning itself. Policy correctness: ARE enforces policy without exception. It does not evaluate whether the policy is correct. A misconfigured policy enforced with perfect fidelity is still a misconfigured policy. Policy design is outside ARE's scope. Policy enforcement is ARE's entire scope. Policy generation is external to ARE and may be derived from regulatory frameworks, human governance boards, or automated synthesis systems operating on compliance requirements. ARE makes whatever policy exists unbypassable. It does not make policy right. Emergent collective behavior: ARE addresses individual agent governance and first- and second-order delegation chains. The Passport and Delegation Model let the control structure compose across agents by preserving identity, authority provenance, and non-amplification at each delegation boundary. This bounds multi-agent authority propagation, but higher-order collective behavior -- behaviors that no individual agent produces but that arise from aggregate action -- remains outside the current model and requires extension of the control structure. The limitation is therefore bounded and extensible, not ignored. Physical side channels: ARE assumes the coprocessor is the only path to consequential action. If an agent can act through channels outside the coprocessor's architectural scope, that action is outside ARE's governance boundary. Scope completeness is a deployment requirement that ARE does not enforce internally. The root of trust: ARE's cryptographic properties depend on the integrity of the key infrastructure underlying Passport issuance and Ledger signing. A compromised root of trust compromises the Passport and Ledger guarantees. ARE assumes the root of trust is secure. Securing it is an infrastructure requirement.

Outcome safety versus action safety: ARE governs action eligibility at the execution boundary. It does not by itself prove that every harmful outcome in a domain can be reduced to an action-level constraint or captured by the implemented policy set. That reduction is a system-specific safety-case task belonging to the surrounding application domain. ARE therefore supplies a control architecture for governed execution, not a universal proof that all downstream losses are eliminated.

12.2 Where ARE Fails Safe

Within its strongest implemented execution path, ARE is designed to fail safe. A Passport that is expired or revoked blocks evaluation. The pre-execution validator rejects stale, unavailable, or replayed context. The action-receipt path blocks atomic execution when receipt generation or receipt-bound Ledger writing fails. The immutable Ledger detects tamper and chain discontinuity. These properties support a fail-closed architecture at the execution boundary.

They should not be read as a proof that every auxiliary Ledger emission path or every distributed deployment topology already carries the same hard-gate guarantee. Some broader fail-safe properties remain partial implementation and future validation work.

13. Implementation Evidence and Deployment Experience

By design, the evidentiary status of current implementations is narrower than full validation. Elements of the architecture have informed regulated financial-services deployment work, and current implementation evidence exists both in ARE itself and in a companion stratified execution substrate. This matters because it shows the design is not merely rhetorical: the controller can be interposed, the authority chain can be verified, receipt-bound execution paths can be implemented, and feedback-carrying evidence can be composed in real software. But these artifacts should not be overread. They do not yet constitute a completed experimental safety case. In the current ARE codebase, passing component suites exist for Passport issuance, pre-execution validation, API gateway behavior, policy coprocessor logic, non-transferable authority verification, action-receipt generation, atomic execution, and immutable Ledger integrity. The governance-strata execution substrate adds passing hook-orchestration and ARE-integration tests around Formation, Authenticity, Measurement, Execution, and Recovery. Together these artifacts support modest claims of buildability, composability, and bounded operational viability. They do not, by themselves, establish hazard completeness, transferable performance bounds, or quantified safety improvement.

13.1 Governance Strata as Pre-Execution Substrate

A current companion implementation composes ARE with independent Formation, Authenticity, and Measurement services upstream of an execution substrate, with Recovery downstream. The architectural distinction matters. Formation tests whether an admissible position can exist at all. Authenticity tests whether the relevant state is real, attributable, and intact. Measurement tests whether the signal is valid and grounded. These strata do not authorize action. They constrain the controller's process model and provide evidence to the commit boundary. Authority remains externalized to ARE at execution time, where the coprocessor determines whether the action is allowed now. This composition is therefore a deployment pattern, not a claim that Formation, Authenticity, Measurement, Execution, or Recovery constitute additional ARE layers or additional STAMP correspondences. It is evidence that ARE can be integrated with a stratified pre-commit substrate without collapsing truth, permission, and execution into one service. Current integration tests in the execution substrate support this narrower claim of composability. What remains open is the formal analysis of which guarantees are preserved under such distributed composition, what coupling assumptions are required, and how upstream failure or latency propagates into the controller's process model.

13.2 Evidence Discipline and Current Gaps

The evidence in this paper falls into four categories: constructive theoretical correspondence, formalizable invariants, implementation evidence, and explicit open questions. No claim should be read more strongly than its evidence class warrants. In the current implementation, a reproducible verification harness reports passing component suites across the core ARE services, and a static interposition audit identifies clearly governed execution paths while still leaving a substantial unknown surface. That is why this paper does not claim enforcement completeness, transferable latency bounds, or experimentally demonstrated safety improvement. A stronger empirical section would require reproducible server-level benchmarks, failure injection, adversarial Passport/Ledger/coprocessor testing, and traceable baselines against weaker governance architectures. Those tasks remain necessary for a stronger experimental paper.

14. Related Work

14.1 STAMP Applications in Software and AI Systems

STAMP has been applied extensively in aerospace, nuclear, medical devices, and transportation. NASA's adoption of STPA for launch vehicle safety analysis and the FAA's incorporation of STAMP-based analysis into avionics guidance represent the framework's deepest engineering adoptions. Leveson's later work increasingly addresses cyber-physical systems where software plays a central role in the control structure. Abdulkhaleq and Wagner (2015) developed XSTAMPP, establishing that STAMP's control structure model extends beyond purely physical processes.

The framework is not inherently physical. It is inherently structural. This is the foundation for its application here. Applications to AI and machine learning exist but operate at a different level of abstraction. Salay, Queiroz, and Czarnecki (2018) apply STPA to machine learning components in autonomous vehicles, treating the ML subsystem as a controlled element within a physical control architecture. This work does not address the governance of autonomous agents acting independently across high-consequence decision boundaries. We are not aware of prior work that establishes STAMP as the explicit theoretical foundation for the governance layer above deployed agent systems.

14.2 Agent Governance Frameworks

The dominant agent governance approaches -- constraint-based, checkpoint-based, and monitoring-based -- share a structural failure when analyzed through STAMP. They are open-loop. They issue constraints or observe behavior but provide no mechanism for the governance system to update its model of what the agent is doing based on observed outcomes. An open-loop governance system cannot maintain process model accuracy. STAMP predicts the failure modes that result. NeMo Guardrails (NVIDIA, 2023) is the most developed enforcement-layer framework currently available. Table 2 in Section 11 provides the full structural comparison. The critical gap: NeMo Guardrails provides stateless output filtering. ARE provides a stateful, interposed, closed-loop control structure. These are different solutions to different problems.

14.3 AI Safety and Alignment

The alignment and interpretability research community addresses a related but distinct problem: ensuring that AI systems behave in accordance with human values and intentions. Constitutional AI (Bai et al., 2022), RLHF (Christiano et al., 2017), and interpretability work operate at the model layer. The distinction is this: alignment asks whether the model will choose to behave safely. ARE asks whether the system will permit unsafe behavior regardless of what the model chooses. Safety engineering's answer to this question has been consistent for fifty years: do not rely on the controlled process choosing to stay within safe boundaries. Build a control structure that enforces boundaries regardless. ARE is that control structure.

15. Conclusion

The problem of governing autonomous agents in high-consequence environments already has a mature theoretical frame. Leveson's STAMP explains what a governance system must contain: a controller, a controlled process, safety constraints, feedback, and a closed loop. This paper argues that ARE provides a constructive, bounded implementation of that frame for autonomous-agent execution governance. The claim is deliberately narrower than formal control equivalence. Within the scope and assumptions stated in Section 4.1, the STAMP mapping and bounded STPA safety case are sufficient for a control argument at the execution boundary; they are not a claim of complete hazard closure, policy correctness, or global multi-agent safety. The Agent Passport, Governance Coprocessor, Operations Ledger, and SDAVL loop together define a control architecture in which authority is explicit, enforcement is interposed, evidence is preserved, and feedback updates the process model after each governed action. Evidence-based control and LLM-MPC are augmenting capabilities rather than pillars of the minimum safety case. Current implementation work, passing component suites, companion substrate tests, and formalizable execution-boundary invariants show that the architecture is buildable and composable, but they do not yet replace the remaining work of broader STPA coverage, adversarial evaluation, machine-checked specification, and reproducible end-to-end measurement. Those limits are stated explicitly because scientific credibility depends on matching claim strength to demonstrated evidence. Intelligence never grants authority. Authority is earned through verifiable policy enforcement.

Current implementation work suggests that admission-time envelopes, scope fidelity checks, behavioral baselines, and lineage-coherence signals are promising process-model inputs, but a unified formal treatment of those signals remains future work.

15.1 Future Work

Five evidence-producing directions follow directly from this work. First, the bounded STPA package presented here should be expanded into domain-specific safety analyses for concrete deployment classes, with fuller causal-scenario coverage and explicit reviewer-facing traceability from losses to tests and evidence artifacts. Second, the minimum execution envelope should be machine-checked in TLA+, PlusCal, or an equivalent temporal-logic formalism. The

key invariants remain no execution without affirmative verdict, no execution with revoked or expired authority, atomicity of execution-boundary evidence, fail-closed behavior under uncertainty, and non-amplifying delegation.

Third, Passport, Ledger, and Coprocessor assumptions should be subjected to adversarial evaluation: replay, revocation lag, evidence mismatch, partial write, bypass attempts, stale reads, degraded-controller behavior, and distributed ordering failures. These are security questions, not merely implementation details. Fourth, the current implementation suites require a broader empirical program: server-level benchmarks, failure injection, baseline comparison against weaker governance architectures, and explicit reporting of what is prevented, detected, or left out of scope.

Fifth, the composition of independent Formation, Authenticity, and Measurement strata with an externalized ARE control plane should be analyzed explicitly as a distributed control architecture with stated timing, integrity, and failure-propagation assumptions. Beyond these validation tasks, later lines of work include a unified formal treatment of global admissibility across lineage and time and an ALI/ALO/ALA-style performance model for measuring whether an agent-governance control structure is operating as intended.

References

Rabanser, S., Kapoor, S., Kirgis, P., Liu, K., Utpala, S., & Narayanan, A. (2026). Towards a Science of AI Agent Reliability. arXiv:2602.16666. Princeton University.

Fernandez, M. (2026). Agent Control Protocol: Admission Control for Agent Actions. arXiv:2603.18829. TraslaIA.

Kaptein, M., Khan, V.-J., & Podstavnychy, A. (2026). Runtime Governance for AI Agents: Policies on Paths. arXiv:2603.16586.

Mazzocchetti, A. M. (2026). Cryptographic Runtime Governance for Autonomous AI Systems: The Aegis Architecture for Verifiable Policy Enforcement. arXiv:2603.16938.

Gaurav, S., Heikkonen, J., & Chaudhary, J. (2025). Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement. arXiv:2508.18765.

Abdulkhaleq, A., & Wagner, S. (2015). XSTAMPP: An eXtensible STAMP Platform as Tool Support for Safety Engineering. STAMP Workshop, MIT.

Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862.

Christiano, P., Leike, J., Brown, T., et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS 2017.

Federal Reserve Board. (2011). SR 11-7: Guidance on Model Risk Management. Board of Governors of the Federal Reserve System.

Kershaw, J. R. M. (2026). Governance Strata: Companion Pre-Execution Substrate Repositories. github.com/srex-dev/governance-strata.

Kershaw, J. R. M. (2026). Agent Responsibility Engineering: Tenets and Foundational Framework. github.com/srex-dev/AgentResponsibilityEngineering.

Leveson, N. (1995). Safeware: System Safety and Computers. Addison-Wesley.

Leveson, N. (2011). Engineering a Safer World: Systems Thinking Applied to Safety. MIT Press.

Leveson, N., & Thomas, J. (2018). STPA Handbook. MIT Partnership for Systems Approaches to Safety and Security.

NIST. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology.

NVIDIA. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. github.com/NVIDIA/NeMo-Guardrails.

Salay, R., Queiroz, R., & Czarnecki, K. (2018). An Analysis of ISO 26262: Using Machine Learning Safely in Automotive Software. SAE Technical Paper.

Thomas, J. (2013). Extending and Automating a Systems-Theoretic Hazard Analysis for Requirements Generation and Analysis. MIT PhD Thesis.